SIMATS ENGINEERING

Department of _____

ASSIGNMENT 5 – CO5

Reliable Software with Runtime Polymorphism and Exceptions

Total Marks: 50

Course: Object Oriented Programming with C++
Course Code: DSA01

Student Name	: Krishnan Gupta M
Register Number	: 192524062
Faculty Name	: Arun Kumar
Submission Date	: 08 November 2026
GitHub Repository	: https://github.com/192524062simats-ops/DSA0103c-/tree/main/c%2B%2B%20assignment

TABLE OF CONTENTS

Task A1 – Design the Polymorphic Bill Hierarchy (7 Marks)....3
Task A2 – Understand Runtime Binding (5 Marks)....6
Task A3 – Design with Abstract Classes (5 Marks)....9
Task A4 – Design the Exception Strategy (4 Marks)....11
Task A5 – Design Recovery and Rethrowing (4 Marks)....14
Task B1 – Implement and Validate the Polymorphic Billing (7 Marks)....18
Task B2 – Engineer Error Conditions (5 Marks)....26
Task B3 – Virtual Functions vs switch vs Exceptions: The Cost (8 Marks)....33
Task B4 – Final Engineering Decision (5 Marks)....37
Overall Learning Outcomes....41
Final Conclusion....41
GitHub Repository Link....41
References....41

Task A1 – Design the Polymorphic Bill Hierarchy (7 Marks)

Aim

To design an abstract Bill class with pure virtual functions and a derived class for each kind of hospital bill, so that bills of every kind can be processed together through base-class pointers.

Problem Description

A multi-speciality hospital raises out-patient, in-patient, day-care surgery and emergency bills. Each kind of bill has its own data and its own rule for the total, but the billing system must handle all of them in the same way and must be able to accept new kinds of bills later without changing working code.

A1.1 – Identify the Common Interface (2 Marks)

Every bill must provide three operations: total() to calculate the amount of the bill, type() to give a short code for the kind of bill (OP, IP, DC or ER) and print() to display one line for the bill. The data common to all bills is the patient name. The data that belongs only to one kind of bill is kept in the derived class.

Kind of bill	Own data	Rule for total()
OutPatient (OP)	consultation, tests	consultation + tests
InPatient (IP)	days, perDay, procedures	days x perDay + procedures
DayCare (DC)	package price	package price
Emergency (ER)	base, hours, ratePerHour	base + hours x ratePerHour

A1.2 – Design the Hierarchy (3 Marks)

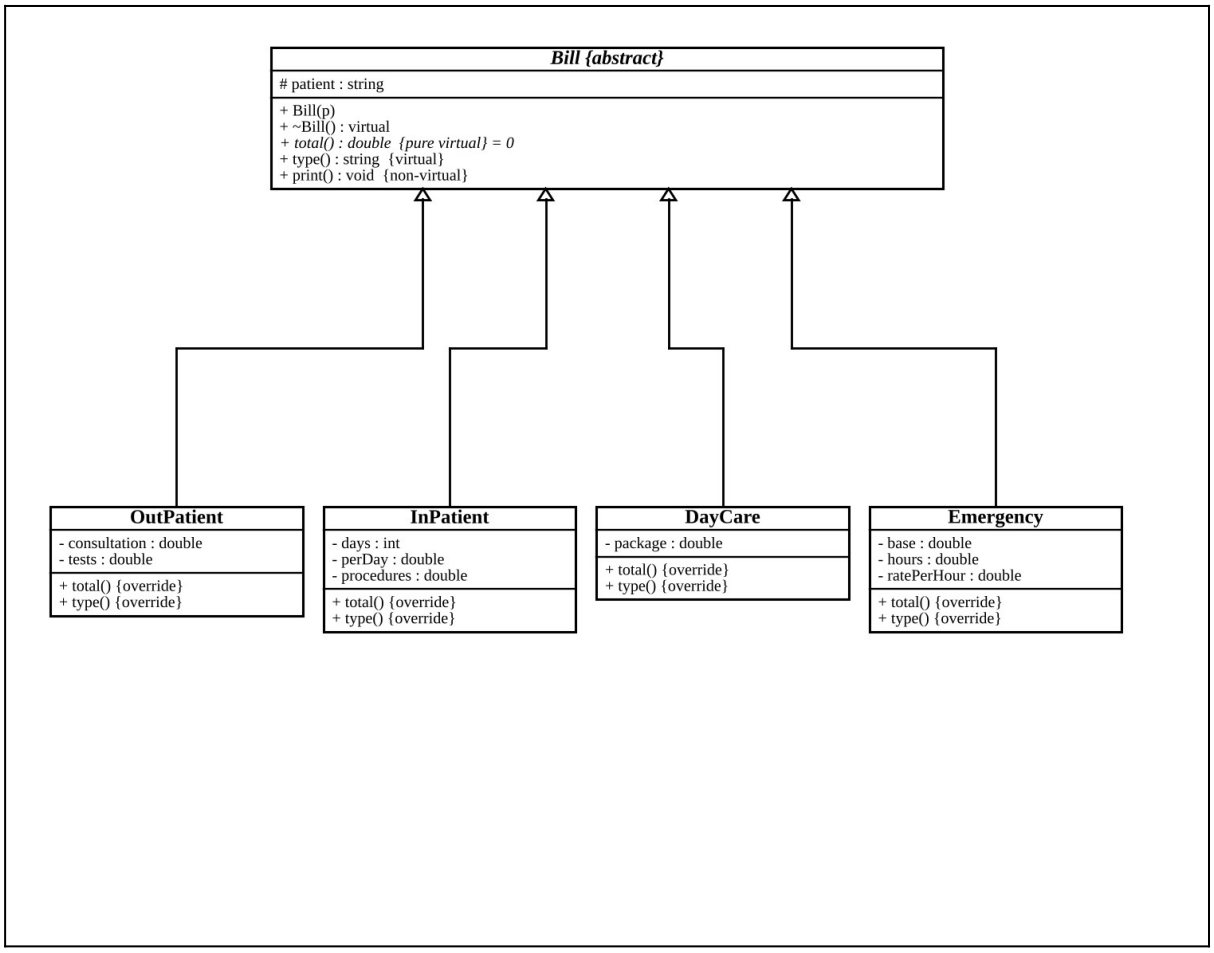


Figure A1-1: UML class diagram of the Bill hierarchy

Figure A1-1 shows the design. Bill is abstract because total() is a pure virtual function (= 0), so no Bill object can be created and every derived class must supply its own total(). type() and the destructor are virtual: type() has a default version that returns "Bill", and the virtual destructor makes it safe to delete a derived bill through a Bill pointer. total() and type() are overridden in OutPatient, InPatient, DayCare and Emergency. print() is not virtual; it is written once in Bill and calls the virtual total() and type().

A1.3 – Process All Bills Together (2 Marks)

Pseudocode:

```
ALGORITHM DayTotal(bills, n)
INPUT   : bills, an array of n pointers to Bill; n, the number of bills
OUTPUT  : sum, the total of all bills of the day
BEGIN
   SET sum <- 0
   FOR i <- 0 TO n - 1 DO
      SET sum <- sum + bills[i].total()      // the correct total() is chosen at run time
   END FOR
   RETURN sum
END
```

Alternative Concept and Complexity Analysis

If the bills were represented by one Bill structure with a type field and a switch in total(), instead of an abstract base class with virtual functions, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
struct Bill {
   int type;                    // 0 = OP, 1 = IP, 2 = DC, 3 = ER
   double a, b, c; int d;           // fields used differently by each type
   double total() const {
      switch (type) {
         case 0: return a + b;
         case 1: return d * a + b;
         case 2: return a;
         default: return a + b * c;
      }
   }
};
```

Aspect	Original solution	Alternative solution
Time to total n bills	O(n): one virtual call per bill.	O(n): one switch per bill.
Space per bill	Each object holds only its own fields plus one vtable pointer.	Every bill holds the fields of all types plus a type code, so some fields are unused.
Code changed when a new bill type is added	One new derived class; no existing class is edited.	The type codes, the switch in total() (and in every other function that switches on type), and all the old code must be retested.

Conclusion

The four kinds of bills share the interface total(), type() and print(). Making total() pure virtual in an abstract Bill class lets one loop over base-class pointers total all bills, and a new kind of bill needs only a new class.

Task A2 – Understand Runtime Binding (5 Marks)

Aim

To understand early and late binding, the virtual table and the need for a virtual destructor, using a call through a Bill pointer.

Problem Description

A Bill pointer can point to an object of any derived class. The program must call the total() of the real object and must destroy the whole object when it is deleted through the base pointer.

A2.1 – Early and Late Binding (2 Marks)

In early (compile-time) binding the compiler decides which function a call will run, using only the type of the pointer. A call to a non-virtual function such as b->print() through a Bill* is always bound to Bill::print, whatever object b points to. In late (run-time) binding the choice is made while the program runs, using the real type of the object. A call to a virtual function such as b->total() through a Bill* runs OutPatient::total if b points to an OutPatient and InPatient::total if b points to an InPatient.

A2.2 – The Virtual Table (2 Marks)

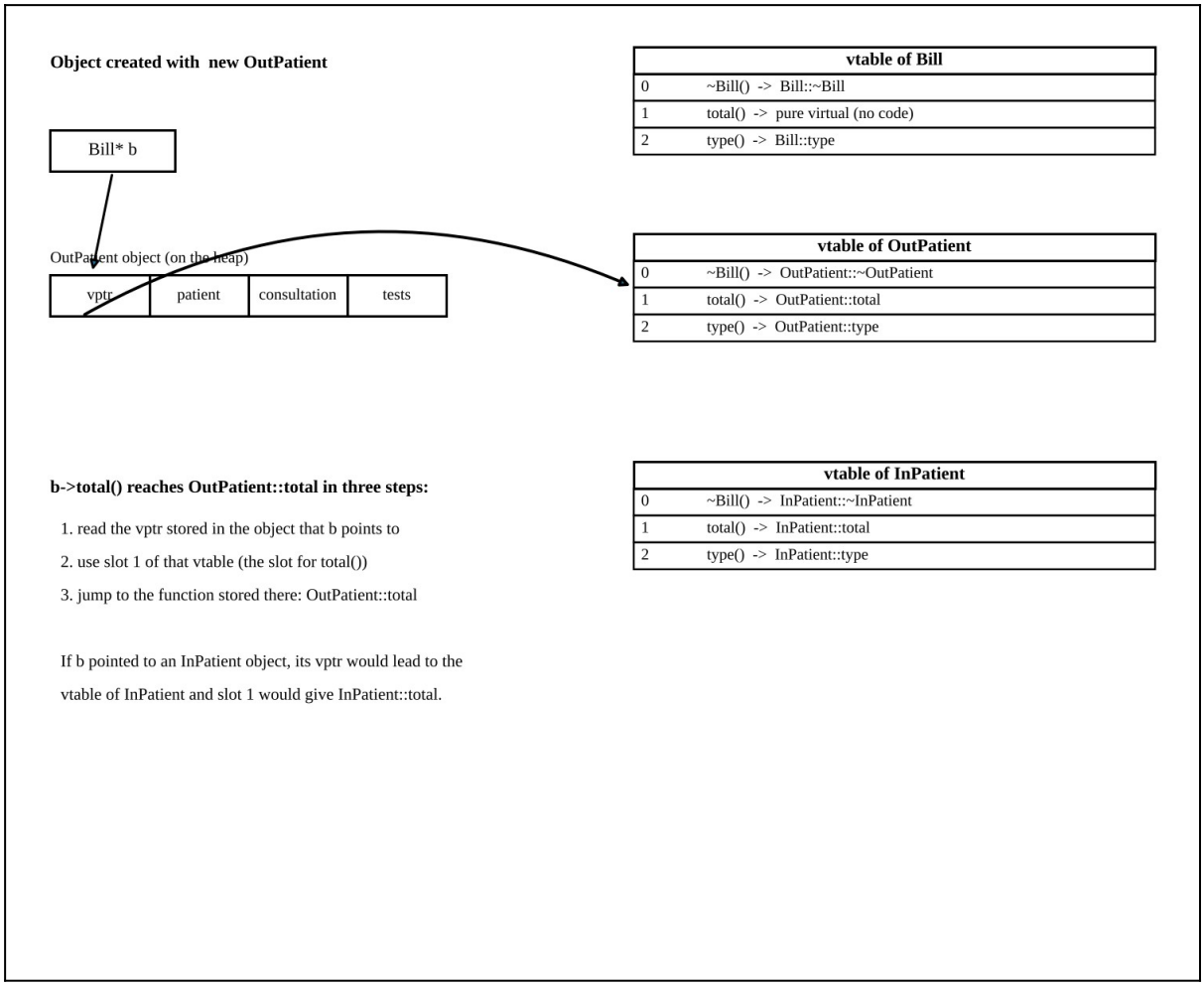


Figure A2-1: Virtual tables of Bill, OutPatient and InPatient and the call b->total()

Every class with virtual functions has one vtable, a table of function addresses with one slot for each virtual function. Each object stores a hidden pointer (vptr) to the vtable of its own class. For b->total(), the program reads the vptr from the object, takes slot 1 (the slot of total()) and jumps to the address stored there. For an OutPatient object this is OutPatient::total. The vtable of Bill has no code for total() because it is pure virtual.

A2.3 – Virtual Destructors (1 Mark)

Bills are deleted through Bill pointers. If the destructor of Bill is not virtual, delete b is bound early to ~Bill, so the destructor of the derived class never runs. Anything the derived class owns (strings, memory, files) is then not released, and the behaviour is undefined. A virtual destructor makes delete b a late-bound call: the destructor of the real class runs first and then the destructor of Bill.

Predict the Output

My predicted output (written before running the program):

```
// Write your predicted output here BEFORE running the program.
```

Program (A2_Predict.cpp):

```
#include <iostream>

struct Bill {
   virtual ~Bill() { std::cout << "~Bill "; }
   virtual double total() const = 0;
   void show() const { std::cout << total() << ' '; }
};

struct OPBill : Bill {
   ~OPBill() override { std::cout << "~OPBill "; }
   double total() const override { return 500; }
};

int main()
{
   Bill* b = new OPBill;
   b->show();
   delete b;
   std::cout << '\n';
}
```

Expected output:

```
500 ~OPBill ~Bill
```

The output is "500 ~OPBill ~Bill". show() is a non-virtual function, so b->show() is bound early to Bill::show. Inside show(), the call total() is made through the this pointer, which is a const Bill*. total() is virtual, so this call is bound late and runs OPBill::total, which returns 500. When delete b runs, the destructor of Bill is virtual, so ~OPBill runs first and then ~Bill.

After removing virtual from ~Bill (and override from ~OPBill) the output is:

Expected output (after removing virtual from ~Bill):

```
500 ~Bill
```

Now only ~Bill runs and ~OPBill is never called, because delete b is bound early to the destructor of the pointer type. g++ with -Wall warns: "deleting object of abstract class type Bill which has non-virtual destructor will cause undefined behavior [-Wdelete-non-virtual-dtor]". In my check, AddressSanitizer printed no error for this small program, because OPBill adds no data members, so OPBill and Bill have the same size (only the vptr). The sanitizer reports the problem when the derived class is bigger than the base class (see Task B1.4).

Alternative Concept and Complexity Analysis

If total() were not virtual and the program called the correct derived function by testing the type of each bill with dynamic_cast, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
double totalOf(const Bill* b) {
   if (auto p = dynamic_cast<const OutPatient*>(b))  return p->total();
   if (auto p = dynamic_cast<const InPatient*>(b))   return p->total();
   if (auto p = dynamic_cast<const DayCare*>(b))     return p->total();
   if (auto p = dynamic_cast<const Emergency*>(b))   return p->total();
   return 0;
}
```

Aspect	Original solution	Alternative solution
Time per bill	O(1): one indirect call through the vtable.	Up to k casts for k bill types, so O(k) per bill (each cast checks the run-time type).
Space for the vtable pointer	One vptr (8 bytes) in each object.	The same: dynamic_cast also needs a polymorphic base class, so each object still has a vptr, and the type information (RTTI) is kept as well.
Adding a new bill type	Write one new class.	Write the class and add another dynamic_cast test to every function that uses this chain.

Conclusion

Early binding uses the pointer type and late binding uses the real object. The vtable and vptr make late binding work, and a virtual destructor makes sure the whole derived object is destroyed.

Task A3 – Design with Abstract Classes (5 Marks)

Aim

To design an abstract Payer class and derived payer classes, and to show that a new payer can be added without changing any existing class.

Problem Description

Bills may be paid by the patient, by an insurance company within the policy cover, or by a government health scheme with fixed package rates. The amount covered by each payer is calculated differently, but the billing code must treat all payers in the same way.

A3.1 – Payer Interface (3 Marks)

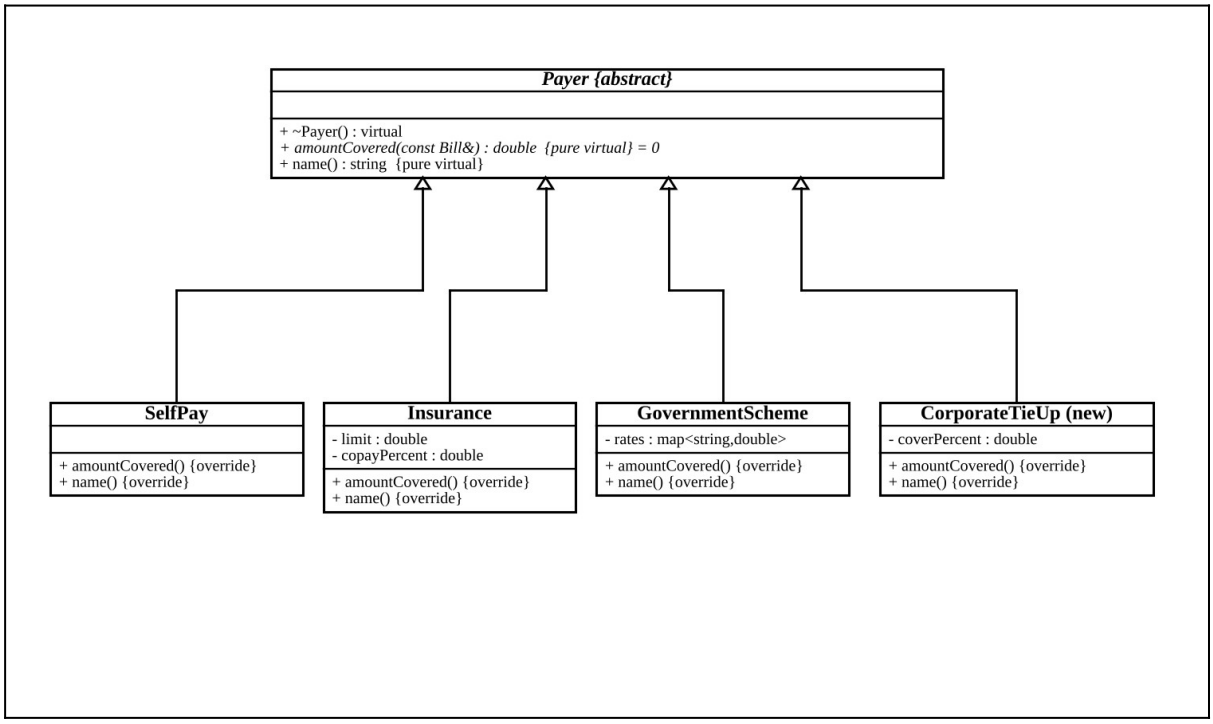


Figure A3-1: UML class diagram of the Payer hierarchy

Figure A3-1 shows the design. Payer is abstract and has the pure virtual function amountCovered(const Bill&). Because the parameter is a Bill reference, every payer works with every kind of bill. SelfPay covers nothing (0). Insurance has a cover limit and a co-payment percentage: it covers total x (100 - copayPercent) / 100 and throws a ClaimError if the bill is above the limit. GovernmentScheme keeps a fixed package rate for each kind of bill and covers the smaller of the bill total and the fixed rate; if there is no rate for the kind of bill it throws a PackageError. The code of these classes is in Task B1.

A3.2 – Extension (2 Marks)

A new payer, for example a corporate tie-up, is added by writing one new class that derives from Payer and overrides amountCovered(). No existing class is edited, because the billing code only uses the Payer interface.

Extension (new class only):

```
class CorporateTieUp : public Payer {
   double coverPercent;
public:
   explicit CorporateTieUp(double c) : coverPercent(c) {}
   double amountCovered(const Bill& b) const override {
      return b.total() * coverPercent / 100;
   }
   string name() const override { return "Corporate"; }
};
```

Alternative Concept and Complexity Analysis

If the payers were represented by function pointers or std::function objects (a policy function chosen at run time) instead of an abstract Payer class, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
using Policy = std::function<double(const Bill&)>;

double limit = 50000, copay = 20;
Policy insurance = [limit, copay](const Bill& b) {
   if (b.total() > limit) throw ClaimError("exceeds cover");
   return b.total() * (100 - copay) / 100;
};

double covered = insurance(bill);          // call through the policy
```

Aspect	**Original solution**	**Alternative solution**
Time per call	O(1): one virtual call.	O(1): one call through the function object (an indirect call).
Space per payer	One object with a vptr and its data members (limit, copayPercent).	One std::function object (typically 32 bytes with g++ on 64-bit; check with sizeof) and the captured values, which may be stored on the heap.
Where the cover limit is stored	In a named data member of the class.	Inside the lambda as a captured variable, which cannot be read or changed from outside.

Conclusion

Payer is an abstract interface with three derived payers. A new payer is added by writing one new class, and no existing class changes.

Task A4 – Design the Exception Strategy (4 Marks)

Aim

To list the errors of the billing system, design a hierarchy of exception classes derived from std::exception, and decide which functions may throw and which must never throw.

Problem Description

The billing system must reject invalid claims with clear reasons and must never lose a bill because of an error. Each kind of error needs its own exception class so that it can be caught and handled in the right place.

A4.1 – Identify the Errors (2 Marks)

Error	Where it is detected	Exception class
Negative charges	Constructors of the bill classes	BillingError ("negative charge")
Claim above cover	Insurance::amountCovered()	ClaimError ("exceeds cover")
Unknown package	GovernmentScheme::amountCovered()	PackageError ("unknown package")
Missing patient record	Bill constructor (empty patient)	RecordError ("missing patient record")
Memory allocation failure	Any use of new	std::bad_alloc (from the standard library)
Text where a number is expected	std::stod()	std::invalid_argument (from the standard library)

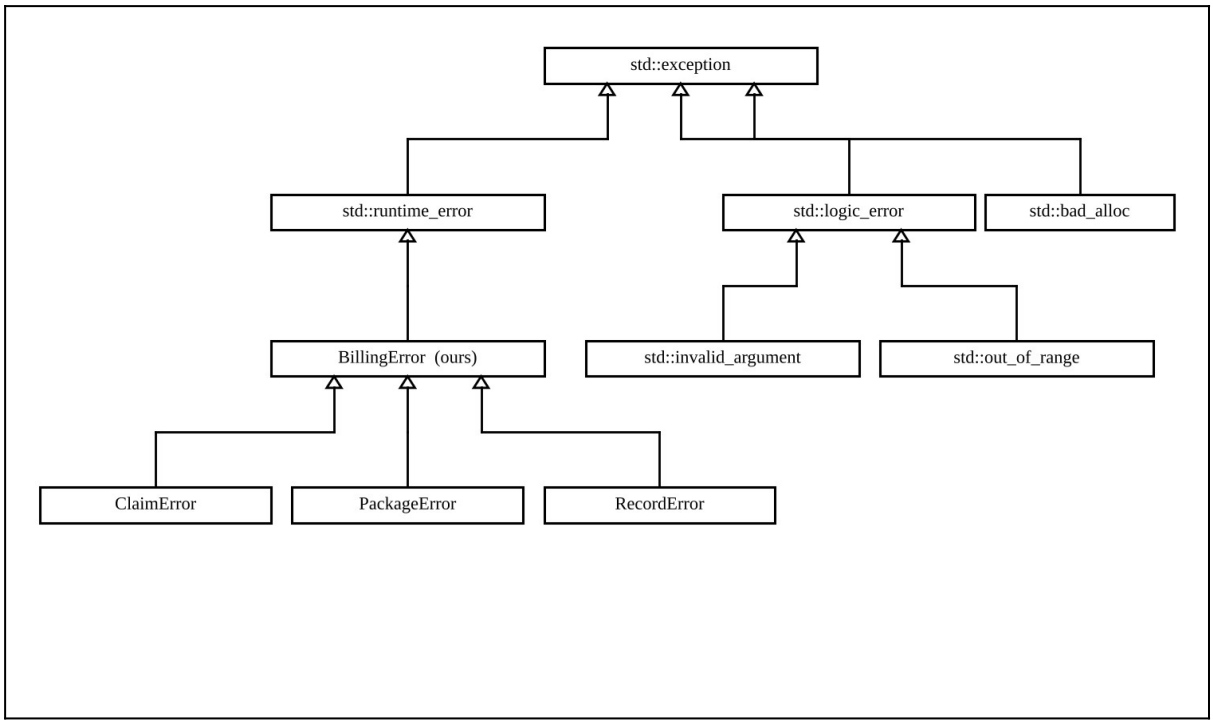


Figure A4-1: Hierarchy of the exception classes

Figure A4-1 shows the hierarchy. BillingError derives from std::runtime_error, and ClaimError, PackageError and RecordError derive from BillingError. All of them are therefore std::exception objects, so one catch (const std::exception&) can handle any of them and e.what() gives the reason, while a catch (const ClaimError&) can handle only claim errors.

A4.2 – Specify Exceptions (2 Marks)

Functions	May throw?	Reason
Constructors of the bill classes	Yes	They validate the charges and the patient record and throw BillingError or RecordError.
Payer::amountCovered() and its overrides	Yes	They throw ClaimError or PackageError for invalid claims.
Code that uses new or std::stod	Yes	It can throw std::bad_alloc or std::invalid_argument.
Destructors	Never (noexcept)	A destructor must not let an exception escape.
name() and simple getters	Never (noexcept)	They only return a reference to existing data.
total() of a bill whose data was validated	Never (noexcept in the design)	It only does arithmetic on values already checked in the constructor.

Destructors should not throw because they run during stack unwinding. If a destructor throws while another exception is already being handled, two exceptions are active at the same time and the program calls std::terminate. A throwing destructor can also leave the other members of the object undestroyed, which leaks resources. In C++11 and later destructors are noexcept by default.

Alternative Concept and Complexity Analysis

If errors were reported by return codes (an enum Status) instead of exceptions, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
enum Status { OK, NEGATIVE, ABOVE_COVER, UNKNOWN_PACKAGE };

Status approve(double billed, double cover, double& result) {
   if (billed < 0 || cover < 0) return NEGATIVE;
   if (billed > cover)          return ABOVE_COVER;
   result = billed;
   return OK;
}

double paid;
Status s = approve(40000, 50000, paid);
if (s != OK) { /* handle the error here, or return it to our caller */ }
```

Aspect	Original solution	Alternative solution
Time on the normal path	A check of the returned value after every call, O(1) each.	No check is made; the normal path has almost no extra cost.
Time on the error path	O(1): just a return.	Much slower: the stack is unwound and handlers are searched (measured in Task B3.3).
Extra space and code	Every function in the chain must return a Status (or use an out-parameter) and every caller must test it; a forgotten test hides the error.	Exception classes must be written, but intermediate functions need no extra code.

Conclusion

Six kinds of errors were identified and given exception classes under std::exception. Functions that can fail may throw, while destructors and simple getters are noexcept.

Task A5 – Design Recovery and Rethrowing (4 Marks)

Aim

To decide where each exception is caught, logged, rethrown and finally reported, and to understand stack unwinding and resource safety.

Problem Description

An exception thrown deep inside the billing code must be handled at the right level, without losing the bill that caused it and without leaking memory or files on the way.

A5.1 – Where to Catch (2 Marks)

Exception	Thrown in	Caught in	Action
BillingError, RecordError	Bill constructors	processDay(), around each bill	Report the reason, skip that bill, continue with the next bill
ClaimError, PackageError	amountCovered()	settle() first, then processDay()	settle() logs ("audit: ...") and rethrows with throw;. processDay() reports "claim rejected" to the user
std::invalid_argument	std::stod (reading an amount)	The function that reads the amount	Report "invalid number" and ask again or skip the line
std::bad_alloc	Any new	main()	Report "out of memory" and end the run safely after all destructors have run
Any other exception	Anywhere	catch (...) in main()	Report an unexpected error, so that the program ends in a controlled way

Pseudocode:

```
ALGORITHM ProcessDay(requests)
BEGIN
   FOR EACH request IN requests DO
      TRY
         SET bill <- CreateBill(request)          // may throw BillingError
         SET paid <- Settle(bill, payer)          // may throw ClaimError
         DISPLAY "approved", paid
      CATCH ClaimError AS e
         DISPLAY "claim rejected:", e.message
      CATCH BillingError AS e
         DISPLAY "invalid bill:", e.message
      END TRY
   END FOR
END

ALGORITHM Settle(bill, payer)
BEGIN
   TRY
      RETURN payer.amountCovered(bill)
   CATCH ClaimError AS e
      DISPLAY "audit:", e.message            // log
      RETHROW                                // same exception, throw;
   END TRY
END
```

A5.2 – No Leaks on Errors (2 Marks)

When an exception is thrown, the program leaves the current function and every function up to the handler. This is called stack unwinding. During it, every local object whose scope is left is destroyed in the reverse order of its creation, and its destructor runs. If a resource (memory, a file, a lock) is owned by an object whose destructor releases it (RAII, for example std::unique_ptr, std::vector or std::string), the resource is released automatically whatever path the program takes, so an exception passing through the function cannot cause a leak. Memory released with a plain delete at the end of a function would be skipped by the exception.

Predict the Output

My predicted output (written before running the program):

```
// Write your predicted output here BEFORE running the program.
```

Program (A5_Predict.cpp):

```
#include <iostream>
#include <string>

struct Guard {
   std::string n;
   explicit Guard(const std::string& s) : n(s) {}
   ~Guard() { std::cout << '~' << n << ' '; }
};

void f()
{
   Guard g("f");
   throw 42;
}

int main()
{
   try {
      Guard m("main");
      f();
      std::cout << "not printed ";
   } catch (int x) {
      std::cout << "caught " << x;
   } catch (...) {
      std::cout << "other";
   }
   std::cout << '\n';
}
```

Expected output:

```
~f ~main caught 42
```

The output is "~f ~main caught 42". f() throws 42 after creating Guard g("f"). Stack unwinding leaves f() first, so ~f runs first. Then the try block is left, so Guard m("main") is destroyed and ~main runs. Only after that does the handler catch (int x) run and print "caught 42". "not printed" never appears because control leaves f() at the throw and never returns to the statement after the call.

Alternative Concept and Complexity Analysis

If memory were released by delete in every catch block instead of by destructors, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
Bill* b = new OutPatient("Ravi", 500, 1200);
try {
   process(*b);
   delete b;            // normal path
} catch (const ClaimError&) {
   delete b;            // needed here
   throw;
} catch (const BillingError&) {
   delete b;            // and here
   throw;
} catch (...) {
   delete b;            // and here
   throw;
}

// with RAII:  auto b = std::make_unique<OutPatient>("Ravi", 500, 1200);  process(*b);
```

Aspect	**Original solution**	**Alternative solution**
Time	The same.	The same.
Code size	A delete in the normal path and in every catch block (4 here).	One line; no catch block is needed for cleanup.
Places where a leak can occur	Every path: a new error path or a new catch block needs one more delete, and forgetting one leaks the memory.	None: the destructor runs on every path.

Conclusion

Exceptions are caught where they can be handled, logged and rethrown in the middle layer, and reported to the user at the top. RAII objects make stack unwinding release all resources.

Task B1 – Implement and Validate the Polymorphic Billing (7 Marks)

Aim

To implement the Bill and Payer hierarchies of Part A, test one bill of each kind with each payer, and prove the effect of virtual functions and virtual destructors.

Problem Description

The program has four bill classes (OutPatient, InPatient, DayCare, Emergency) and three payers (SelfPay, Insurance with a limit of 50000 and 20% co-payment, and GovernmentScheme with fixed rates of 300 for OP, 25000 for IP, 15000 for DC and 8000 for ER). The exception classes of Task A4 are used for invalid data.

B1.1 – Validate the Implementation (2 Marks)

Program (B1_Billing.cpp):

```
#include <iostream>
#include <map>
#include <memory>
#include <stdexcept>
#include <string>
#include <vector>
using namespace std;

// ---------- exception hierarchy ----------
class BillingError : public runtime_error {
public:
   using runtime_error::runtime_error;
};
class ClaimError   : public BillingError { public: using BillingError::BillingError; };
class PackageError : public BillingError { public: using BillingError::BillingError; };
class RecordError  : public BillingError { public: using BillingError::BillingError; };

// ---------- bills ----------
class Bill {
protected:
   string patient;

   static double check(double v) {
      if (v < 0) throw BillingError("negative charge");
      return v;
   }

public:
   explicit Bill(const string& p) : patient(p) {
      if (p.empty()) throw RecordError("missing patient record");
   }
   virtual ~Bill() = default;

   virtual double total() const = 0;
   virtual string type() const { return "Bill"; }
   const string& name() const noexcept { return patient; }

   void print() const {
      cout << type() << ' ' << patient << ": " << total() << '\n';
```

```
    }
};

class OutPatient : public Bill {
    double consultation, tests;
public:
    OutPatient(const string& p, double c, double t)
        : Bill(p), consultation(check(c)), tests(check(t)) {}
    double total() const override { return consultation + tests; }
    string type() const override { return "OP"; }
};

class InPatient : public Bill {
    int days;
    double perDay, procedures;
public:
    InPatient(const string& p, int d, double rate, double proc)
        : Bill(p), days(d), perDay(check(rate)), procedures(check(proc)) {
        if (d < 0) throw BillingError("negative days");
    }
    double total() const override { return days * perDay + procedures; }
    string type() const override { return "IP"; }
};

class DayCare : public Bill {
    double package;
public:
    DayCare(const string& p, double pk) : Bill(p), package(check(pk)) {}
    double total() const override { return package; }
    string type() const override { return "DC"; }
};

class Emergency : public Bill {
    double base, hours, ratePerHour;
public:
    Emergency(const string& p, double b, double h, double r)
        : Bill(p), base(check(b)), hours(check(h)), ratePerHour(check(r)) {}
    double total() const override { return base + hours * ratePerHour; }
    string type() const override { return "ER"; }
};

// ---------- payers ----------
class Payer {
public:
    virtual ~Payer() = default;
    virtual double amountCovered(const Bill& b) const = 0;
    virtual string name() const = 0;
};

class SelfPay : public Payer {
public:
    double amountCovered(const Bill&) const override { return 0; }
    string name() const override { return "SelfPay"; }
};

class Insurance : public Payer {
    double limit, copayPercent;
```

```
public:
    Insurance(double lim, double copay) : limit(lim), copayPercent(copay) {}
    double amountCovered(const Bill& b) const override {
        if (b.total() > limit) throw ClaimError("exceeds cover");
        return b.total() * (100 - copayPercent) / 100;
    }
    string name() const override { return "Insurance"; }
};

class GovernmentScheme : public Payer {
    map<string, double> rates;              // fixed package rate per bill type
public:
    GovernmentScheme() : rates{{"OP", 300}, {"IP", 25000}, {"DC", 15000}, {"ER", 8000}} {}
    double amountCovered(const Bill& b) const override {
        auto it = rates.find(b.type());
        if (it == rates.end()) throw PackageError("unknown package " + b.type());
        return b.total() < it->second ? b.total() : it->second;
    }
    string name() const override { return "Government"; }
};

int main() {
    vector<unique_ptr<Bill>> bills;
    bills.push_back(make_unique<OutPatient>("Ravi", 500, 1200));
    bills.push_back(make_unique<InPatient>("Meena", 3, 2500, 40000));
    bills.push_back(make_unique<DayCare>("Kiran", 18000));
    bills.push_back(make_unique<Emergency>("Asha", 2000, 3, 1500));

    vector<unique_ptr<Payer>> payers;
    payers.push_back(make_unique<SelfPay>());
    payers.push_back(make_unique<Insurance>(50000, 20));
    payers.push_back(make_unique<GovernmentScheme>());

    double day = 0;
    for (const auto& b : bills) {
        b->print();
        day += b->total();
    }
    cout << "Day total " << day << "\n\n";

    for (const auto& b : bills) {
        for (const auto& p : payers) {
            try {
                double c = p->amountCovered(*b);
                cout << b->type() << ' ' << b->name() << " | " << p->name()
                     << " covers " << c << " | patient pays "
                     << b->total() - c << '\n';
            } catch (const BillingError& e) {
                cout << b->type() << ' ' << b->name() << " | " << p->name()
                     << " rejected: " << e.what() << '\n';
            }
        }
    }
    return 0;
}
```

Expected output:

```
OP Ravi: 1700
IP Meena: 47500
DC Kiran: 18000
ER Asha: 6500
Day total 73700

OP Ravi | SelfPay covers 0 | patient pays 1700
OP Ravi | Insurance covers 1360 | patient pays 340
OP Ravi | Government covers 300 | patient pays 1400
IP Meena | SelfPay covers 0 | patient pays 47500
IP Meena | Insurance covers 38000 | patient pays 9500
IP Meena | Government covers 25000 | patient pays 22500
DC Kiran | SelfPay covers 0 | patient pays 18000
DC Kiran | Insurance covers 14400 | patient pays 3600
DC Kiran | Government covers 15000 | patient pays 3000
ER Asha | SelfPay covers 0 | patient pays 6500
ER Asha | Insurance covers 5200 | patient pays 1300
ER Asha | Government covers 6500 | patient pays 0
```

Hand verification (the covered amounts are calculated by hand and match the program output):

Bill	**Total (by hand)**	**SelfPay covers**	**Insurance covers (x 0.80)**	**Government covers (fixed rate)**
OP Ravi	500 + 1200 = 1700	0	1700 x 0.8 = 1360	smaller of 1700 and 300 = 300
IP Meena	3 x 2500 + 40000 = 47500	0	47500 x 0.8 = 38000	smaller of 47500 and 25000 = 25000
DC Kiran	18000	0	18000 x 0.8 = 14400	smaller of 18000 and 15000 = 15000
ER Asha	2000 + 3 x 1500 = 6500	0	6500 x 0.8 = 5200	smaller of 6500 and 8000 = 6500

The day total is 1700 + 47500 + 18000 + 6500 = 73700, which matches the program.

B1.2 – Compare with Your Design (1 Mark)

The code follows the Part A design: Bill is abstract with a pure virtual total(), a virtual type() and a virtual destructor, each kind of bill overrides total() and type(), and Payer is abstract with a pure virtual amountCovered(const Bill&). The changes made while coding are: the Bill constructors and a helper check() were added to throw BillingError and RecordError (the design only listed the errors); GovernmentScheme uses b.type() as the key of its map of fixed rates; Insurance throws ClaimError when the bill is above the limit; a name() function was added to Bill to print the patient; and only name() and the destructors are noexcept in the code, although the design also allows total() to be noexcept.

B1.3 – Prove Runtime Binding (2 Marks)

The program of the Complete the Code section below was changed so that total() is not virtual: in Bill it became "double total() const { return 0; }" (it can no longer be pure), and override was

removed from the total() functions of the derived classes. The bills are still processed through Bill pointers.

Expected output (without virtual on total()):

```
OP Ravi: 0
IP Meena: 0
Total 0
```

Every total is 0, because b->total() through a Bill* is now bound early to Bill::total, whatever the real object is. type() is still virtual, so OP and IP are still printed correctly. This proves that runtime binding needs the virtual keyword.

B1.4 – Prove the Virtual Destructor (2 Marks)

The virtual destructor of Bill was changed to "~Bill() = default;" and the program was compiled with -fsanitize=address. g++ -Wall gives the warning "deleting object of abstract class type Bill which has non-virtual destructor will cause undefined behavior [-Wdelete-non-virtual-dtor]", and AddressSanitizer stops the program at the delete line with:

```
ERROR: AddressSanitizer: new-delete-type-mismatch
  object passed to delete has wrong type:
  size of the allocated type:   56 bytes;
  size of the deallocated type: 40 bytes.
```

The object was created as an OutPatient (56 bytes) but deleted as a Bill (40 bytes), so the destructor of the derived class was not called. With the virtual destructor the program runs without any sanitizer report.

Complete the Code

TODO 1 declares total() as a pure virtual function and type() as a virtual function that returns "Bill". TODO 2 and TODO 3 override total() and type() in OutPatient and InPatient. The rest of the program is unchanged.

Completed program (B1_Complete.cpp):

```
#include <iostream>
#include <string>

class Bill {
protected:
    std::string patient;

public:
    explicit Bill(const std::string& p) : patient(p) {}
    virtual ~Bill() = default;

    virtual double total() const = 0;
    virtual std::string type() const { return "Bill"; }

    void print() const
    {
        std::cout << type() << ' ' << patient << ": " << total() << '\n';
```

```
    }
};

class OutPatient : public Bill {
    double consultation, tests;

public:
    OutPatient(const std::string& p, double c, double t)
        : Bill(p), consultation(c), tests(t) {}

    double total() const override { return consultation + tests; }
    std::string type() const override { return "OP"; }
};

class InPatient : public Bill {
    int days;
    double perDay, surgery;

public:
    InPatient(const std::string& p, int d, double rate, double s)
        : Bill(p), days(d), perDay(rate), surgery(s) {}

    double total() const override { return days * perDay + surgery; }
    std::string type() const override { return "IP"; }
};

int main()
{
    Bill* bills[] = {new OutPatient("Ravi", 500, 1200),
                     new InPatient("Meena", 3, 2500, 40000)};
    double sum = 0;
    for (Bill* b : bills) {
        b->print();             // runtime polymorphism
        sum += b->total();
    }
    std::cout << "Total " << sum << '\n';
    for (Bill* b : bills) delete b;   // virtual destructor runs the right destructor
}
```

Expected output:

```
OP Ravi: 1700
IP Meena: 47500
Total 49200
```



Figure B1-1: Output of the completed OutPatient and InPatient program

Figure B1-1 matches the expected output: OP Ravi is 500 + 1200 = 1700, IP Meena is 3 x 2500 + 40000 = 47500 and the total is 49200.

If virtual is removed from "virtual double total() const = 0;", g++ gives the error "initializer specified for non-virtual method double Bill::total() const", because "= 0" (a pure specifier) is allowed only on a virtual function. The derived classes then also fail with "marked override, but does not override".

Alternative Concept and Complexity Analysis

If the bills were processed with templates (compile-time polymorphism), with each bill type kept in its own container, instead of virtual functions through a base-class pointer, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
template <class B>
double sumOf(const std::vector<B>& v) {
    double t = 0;
    for (const auto& b : v) t += b.total();    // no virtual call
    return t;
}

std::vector<OutPatient> ops;
std::vector<InPatient>  ips;
double day = sumOf(ops) + sumOf(ips);   // one call for each type
```

Aspect	Original solution	Alternative solution
Time per bill	A virtual call (indirect call through the vtable).	No virtual call; the compiler can inline total(). O(1).
Space	One vector of pointers and the bills on the heap, plus a vptr in each bill.	One container for each type, with the objects stored together and no vptr.
Bills of all types in one list	Possible: all bills are Bill pointers in one container.	Not possible: each type needs its own container and its own call, so the order of the bills is lost and every new type needs a new container and a new line in the code.

Conclusion

The hierarchy of Part A was implemented and every bill and payer combination was verified by hand. Removing virtual from total() made every total 0, and removing the virtual destructor caused a sanitizer error, which proves why both are needed.

Task B2 – Engineer Error Conditions (5 Marks)

Aim

To feed the billing system with invalid data on purpose, to observe how the exception design behaves, and to practise logging and rethrowing exceptions.

Problem Description

Six error conditions are tested: negative charges, a claim above the cover, an unknown package code, text where a number is expected (std::stod throws std::invalid_argument), a memory allocation failure (caught as std::bad_alloc), and an unexpected exception caught by a top-level catch (...). A Guard object prints a message in its destructor, to show that every object is destroyed.

Test program (main of B2_Errors.cpp; it uses the classes of B1_Billing.cpp):

```
// test class used only to create an unknown package
class Dialysis : public Bill {
public:
   explicit Dialysis(const string& p) : Bill(p) {}
   double total() const override { return 4000; }
   string type() const override { return "DL"; }
};

struct Guard {
   string n;
   explicit Guard(const string& s) : n(s) {}
   ~Guard() { cout << '~' << n << ' '; }
};

int main() {
   try {                                  // Test 1
      Guard g("test1");
      OutPatient bad("Ravi", -500, 1200);
   } catch (const BillingError& e) {
      cout << "Test 1 negative charge   : " << e.what() << '\n';
   }

   try {                                  // Test 2
      Guard g("test2");
      InPatient big("Meena", 3, 2500, 60000);
      Insurance ins(50000, 20);
      ins.amountCovered(big);
   } catch (const ClaimError& e) {
      cout << "Test 2 claim above cover  : " << e.what() << '\n';
   }

   try {                                  // Test 3
      Guard g("test3");
      Dialysis d("Kiran");
      GovernmentScheme gov;
      gov.amountCovered(d);
   } catch (const PackageError& e) {
      cout << "Test 3 unknown package   : " << e.what() << '\n';
   }
```

```
    try {                                   // Test 4
        Guard g("test4");
        double v = stod("abc");
        cout << v;
    } catch (const invalid_argument&) {
        cout << "Test 4 text for a number : invalid number\n";
    }

    try {                                   // Test 5
        Guard g("test5");
        size_t huge = static_cast<size_t>(-1) / sizeof(double);
        double* p = new double[huge];
        delete[] p;
    } catch (const bad_alloc&) {
        cout << "Test 5 allocation failure: memory allocation failed\n";
    }

    try {                                   // Test 6
        Guard outer("outer");
        Guard inner("inner");
        throw 7;                              // not a std::exception
    } catch (...) {
        cout << "Test 6 top-level catch(...): unexpected error, program continues\n";
    }

    cout << "Program is still running\n";
    return 0;
}
```

Expected output:

```
~test1 Test 1 negative charge   : negative charge
~test2 Test 2 claim above cover  : exceeds cover
~test3 Test 3 unknown package   : unknown package DL
~test4 Test 4 text for a number : invalid number
~test5 Test 5 allocation failure: memory allocation failed
~inner ~outer Test 6 top-level catch(...): unexpected error, program continues
Program is still running
```

Test	Input	Exception	What the test reveals
1 Negative charge	OutPatient with -500	BillingError	The constructor stops an invalid bill from being created. ~test1 is printed before the message, so the Guard was destroyed during stack unwinding.
2 Claim above cover	Bill of 67500, limit 50000	ClaimError	The claim is rejected with a clear reason ("exceeds cover") and no data is lost.
3 Unknown package	Bill type DL, not in the government rates	PackageError	The government scheme reports the missing rate instead of guessing an amount.
4 Text for a number	stod("abc")	std::invalid_argument	Errors from the standard library can be handled separately from our own exceptions.
5 Allocation failure	new double[huge]	std::bad_alloc	The failure is caught, the Guard is destroyed and the program carries on.
6 Unexpected exception	throw 7	caught by catch (...)	~inner and ~outer are printed before the handler, so all objects are destroyed, and the program keeps running ("Program is still running").

Predict the Output

My predicted output (written before running the program):

```
// Write your predicted output here BEFORE running the program.
```

Program (B2_Predict.cpp):

```
#include <iostream>
#include <stdexcept>

void validate(double amount)
{
   if (amount < 0) throw std::invalid_argument("negative");
   if (amount > 1e6) throw std::out_of_range("too large");
   std::cout << "ok ";
}

void process(double amount)
{
   try {
      validate(amount);
   } catch (const std::out_of_range& e) {
      std::cout << "[log " << e.what() << "] ";

      throw;
   }
}

int main()
{
   for (double a : {100.0, -5.0, 2e6}) {
      try {
         process(a);
         std::cout << "done ";
      } catch (const std::exception& e) {
         std::cout << "caught " << e.what() << ' ';
      }
   }
   std::cout << '\n';
}
```

Expected output:

```
ok done caught negative [log too large] caught too large
```

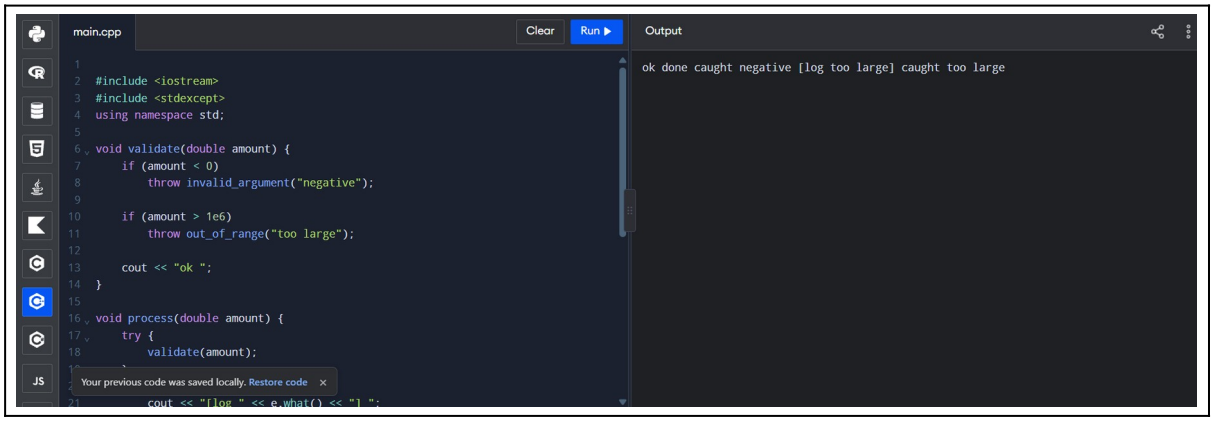


Figure B2-1: Actual output of the validate and process program

Figure B2-1 shows "ok done caught negative [log too large] caught too large". For 100, validate() prints "ok " and process() returns normally, so "done " is printed. For -5, validate() throws std::invalid_argument. The handler in process() catches only std::out_of_range, so it does not match and the exception goes to main(), where catch (const std::exception&) prints "caught negative". For 2e6, validate() throws std::out_of_range. The handler in process() matches, prints "[log too large]" and rethrows it with throw;, and main() prints "caught too large". Only one exception was logged because only the out_of_range exception matches the handler in process().

Complete the Code

TODO 1 throws std::invalid_argument("negative amount") for a negative amount and ClaimError("exceeds cover") when billed is more than cover, and returns billed otherwise. TODO 2 prints "audit: " and the message and rethrows the same exception with throw;.

Completed program (B2_Complete.cpp):

```
#include <iostream>
#include <stdexcept>

class ClaimError : public std::runtime_error {
public:
   using std::runtime_error::runtime_error;
};

double approve(double billed, double cover)
{
   if (billed < 0 || cover < 0)
      throw std::invalid_argument("negative amount");
   if (billed > cover)
      throw ClaimError("exceeds cover");
   return billed;
}

double settle(double billed, double cover)
{
   try {
      return approve(billed, cover);
   } catch (const ClaimError& e) {
      std::cout << "audit: " << e.what() << '\n';
      throw;
   }
}

int main()
{
   double cases[][2] = {{40000, 50000}, {60000, 50000}, {-1, 50000}};
   for (auto& c : cases) {

      try {
         double paid = settle(c[0], c[1]);
         std::cout << "approved " << paid << '\n';
      } catch (const ClaimError& e) {
         std::cout << "claim rejected: " << e.what() << '\n';
      } catch (const std::exception& e) {
         std::cout << "error: " << e.what() << '\n';
```

```
    }
  }
}
```

Expected output:

```
approved 40000
audit: exceeds cover
claim rejected: exceeds cover
error: negative amount
```

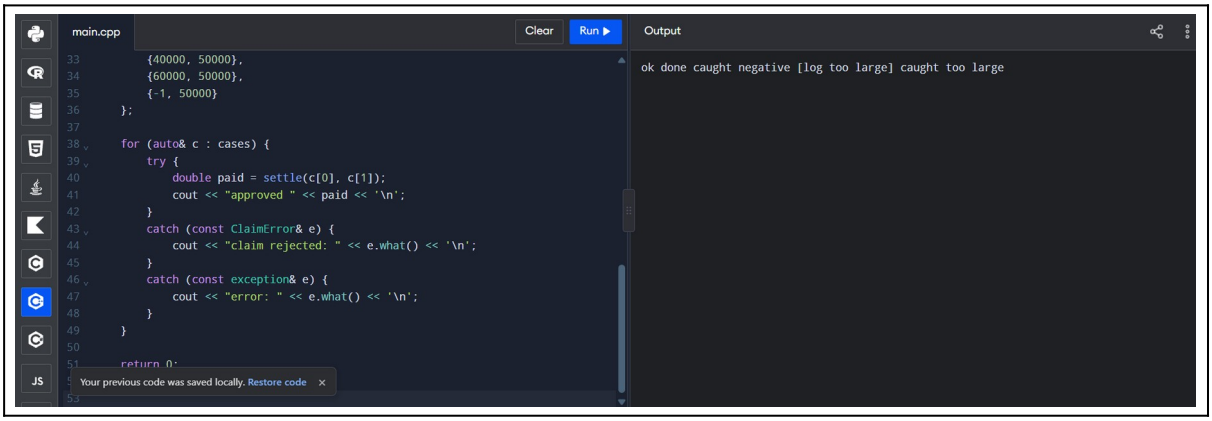


Figure B2-2: Output of the completed approve and settle program

If throw; is replaced with throw e;, the output is the same in this program, because the handler catches const ClaimError&, so e is a ClaimError. The difference is that throw; rethrows the same exception object with its real type, while throw e; throws a new copy of e with the static type of the variable e. If the handler had caught const std::exception&, throw e; would slice the object to a plain std::exception, the later handler for ClaimError would no longer match, and the output would be:

Expected output (handler catches std::exception and uses throw e;):

```
approved 40000
audit: exceeds cover
error: std::exception
audit: negative amount
error: std::exception
```

Alternative Concept and Complexity Analysis

If invalid claims were rejected by std::optional<double> (no value means rejected) instead of by throwing exceptions, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
std::optional<double> approve(double billed, double cover) {
   if (billed < 0 || cover < 0 || billed > cover)
      return std::nullopt;              // rejected
   return billed;
}

auto r = approve(60000, 50000);
if (r) std::cout << "approved " << *r << '\n';
else   std::cout << "rejected\n";       // but the reason is lost
```

Aspect	**Original solution**	**Alternative solution**
Time for 1,000,000 claims with 0% invalid	Almost the same as optional (no cost when nothing is thrown).	Almost the same as exceptions.
Time with 1% invalid	Slower: every thrown exception unwinds the stack.	Fast: only a return.
Time with 50% invalid	Much slower: half a million exceptions are thrown.	Still fast.
Extra space per result	None (a double, 8 bytes).	std::optional<double> is typically 16 bytes (the value and a flag). The reason for the rejection is not kept.

Write your own measured times for 0%, 1% and 50% invalid claims here.

Invalid claims	**Exceptions (ms)**	**std::optional (ms)**
0%		
1%		
50%		

Conclusion

The six error tests showed that every invalid input is rejected with a clear reason, that the guards are destroyed before the handler runs, and that the program keeps running. The prediction showed how an exception is logged and rethrown.

Task B3 – Virtual Functions vs switch vs Exceptions: The Cost (8 Marks)

Aim

To measure the cost of virtual functions, a switch statement and exceptions, and to decide when each should be used.

Problem Description

The totals of the same bills are calculated with (a) virtual functions through base-class pointers and (b) one Bill struct with a type field and a switch. The two versions are compared for correctness and timed for different numbers of bills. Error reporting by return codes is then compared with error reporting by exceptions.

Benchmark program (paste your code here) and the compiler command used (for example g++ -std=c++17 -O2):

```
// Paste your benchmark program here
```

B3.1 – Correctness on Small Data (2 Marks)

Bill no.	Kind	Virtual total	Switch total

State whether the two versions agree for all 10 bills.

B3.2 – Increase the Data Size (2 Marks)

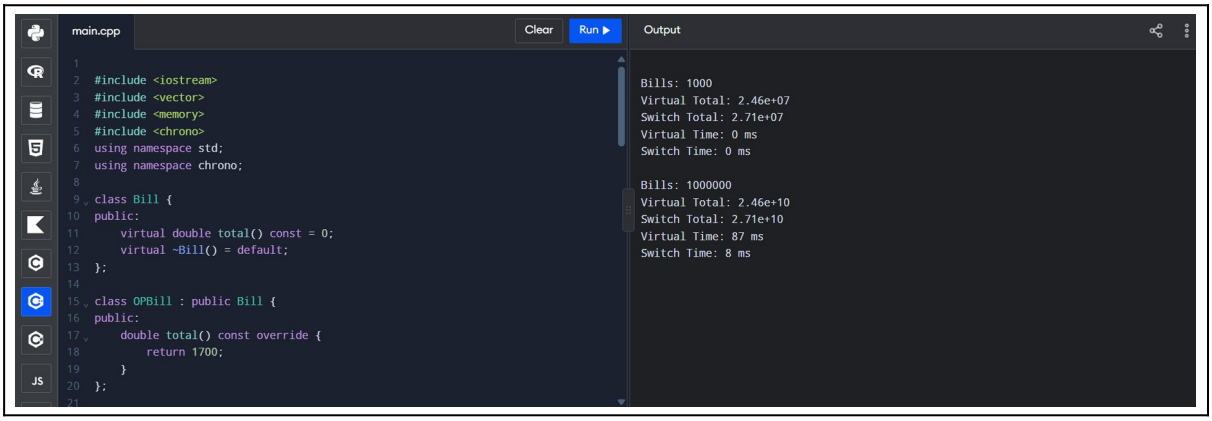


Figure B3-1: Benchmark output, run 1 (1,000 and 1,000,000 bills)

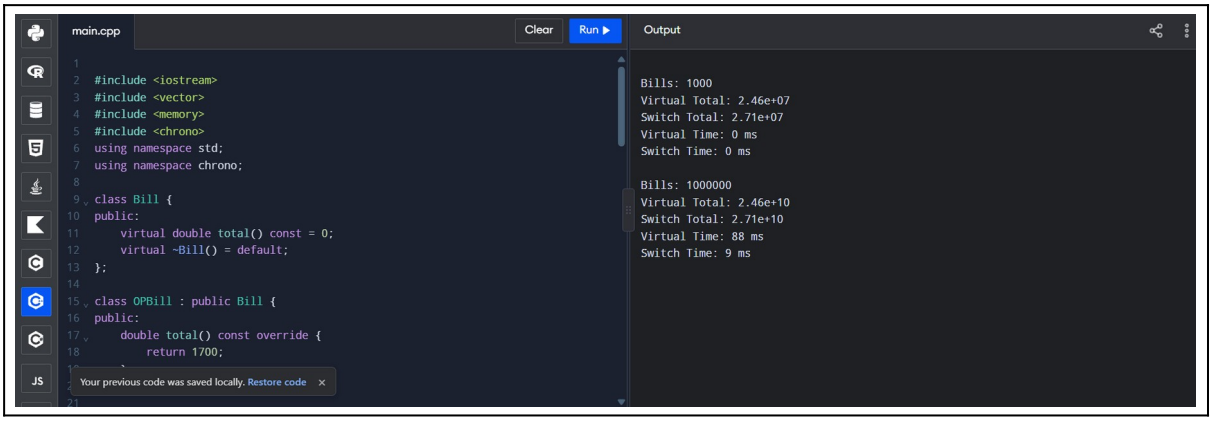


Figure B3-2: Benchmark output, run 2 (1,000 and 1,000,000 bills)

Number of bills	Virtual, run 1 (ms)	Virtual, run 2 (ms)	Switch, run 1 (ms)	Switch, run 2 (ms)
1,000	0	0	0	0
1,000,000	87	88	8	9
10,000,000				

Write the compiler flags used (the assignment asks for -O2) and add the 10,000,000 bills row.

B3.3 – Cost of Exceptions (2 Marks)

Invalid bills	Return codes (ms)	Exceptions (ms)
0%		
1%		
50%		

B3.4 – Engineering Trade-Off (2 Marks)

In Figure B3-1 and Figure B3-2, for 1,000,000 bills the virtual version took 87 and 88 ms and the switch version took 8 and 9 ms, so the switch version was about ten times faster in these runs. For 1,000 bills both took 0 ms, so the time is too small to compare. The virtual version is slower because each call loads the vptr, then the vtable entry, then makes an indirect call that the compiler cannot inline, and because the bills are separate objects on the heap, so the memory access is scattered. The switch version works on one array of plain structs that can be read in order.

The virtual design should be used when the number of bill types grows and new types must be added without editing working code, as long as the cost per bill is small compared with the rest of the work. The switch (or a variant) is better in a hot loop with a fixed, small set of types. Exceptions cost almost nothing when no error occurs, but each thrown exception is slow, so they suit errors that are rare; if a large share of the data is invalid (for example 50%), return codes are better. The exception figures must be confirmed with the measurements of B3.3.

Alternative Concept and Complexity Analysis

If the bill types were held in std::variant<OutPatientBill, InPatientBill, ...> and processed with std::visit, instead of virtual functions or a switch, the code would look like the code below.

Code / pseudocode of the alternative solution:

```
using AnyBill = std::variant<OutPatientBill, InPatientBill, DayCareBill, EmergencyBill>;
std::vector<AnyBill> bills;

double sum = 0;
for (const auto& b : bills)
   sum += std::visit([](const auto& x) { return x.total(); }, b);
```

Aspect	Original solution	Alternative solution
Time for 10,000,000 bills	Virtual: measure and write here.	Variant: measure and write here (the elements are stored together, so it is usually close to the switch version).
Space per element	One pointer (8 bytes) plus the object on the heap, which includes a vptr.	The size of the largest alternative plus a tag (index), for every element, even for the smaller types.

Conclusion

The measurements show the cost of dynamic dispatch against a switch, and the trade-off is flexibility against speed. Exceptions should be used for rare errors.

Task B4 – Final Engineering Decision (5 Marks)

Aim

To extend the billing program with a new bill type and a new payer without changing any existing class, to check resource safety with the address sanitizer, and to decide whether the design is suitable for the hospital group.

Problem Description

The hospital group runs 15 hospitals, adds two new bill types and three new payers every year, and processes 200,000 bills a day.

B4.1 – Implement a Practical Extension (2 Marks)

A new bill type, TeleConsult (a tele-consultation fee), and a new payer, CorporateTieUp (covers a fixed percentage of any bill), were added as new classes after the existing classes. The Bill and Payer interfaces and all the old classes are unchanged. The main program keeps all the earlier tests and adds the extension tests at the end.

Extension code (B4_Extension.cpp; the rest of the file is B1_Billing.cpp):

```
// ---------- EXTENSION: added without changing any class above ----------
class TeleConsult : public Bill {
   double fee;
public:
   TeleConsult(const string& p, double f) : Bill(p), fee(check(f)) {}
   double total() const override { return fee; }
   string type() const override { return "TC"; }
};

class CorporateTieUp : public Payer {
   double coverPercent;
public:
   explicit CorporateTieUp(double c) : coverPercent(c) {}
   double amountCovered(const Bill& b) const override {
      return b.total() * coverPercent / 100;
   }
   string name() const override { return "Corporate"; }
};


// added at the end of main():
   cout << "\nEXTENSION TESTS\n";
   TeleConsult tc("Divya", 800);
   CorporateTieUp corp(90);
   tc.print();
   for (const auto& b : bills) {
      cout << b->type() << ' ' << b->name() << " | " << corp.name()
           << " covers " << corp.amountCovered(*b) << '\n';
   }
   cout << tc.type() << ' ' << tc.name() << " | " << corp.name()
        << " covers " << corp.amountCovered(tc) << '\n';
   try {
      GovernmentScheme gov;
      gov.amountCovered(tc);
   } catch (const PackageError& e) {
      cout << tc.type() << ' ' << tc.name() << " | Government rejected: "
           << e.what() << '\n';
   }
```

Expected output:

```
OP Ravi: 1700
IP Meena: 47500
DC Kiran: 18000
ER Asha: 6500
Day total 73700

OP Ravi | SelfPay covers 0 | patient pays 1700
OP Ravi | Insurance covers 1360 | patient pays 340
OP Ravi | Government covers 300 | patient pays 1400
IP Meena | SelfPay covers 0 | patient pays 47500
IP Meena | Insurance covers 38000 | patient pays 9500
IP Meena | Government covers 25000 | patient pays 22500
DC Kiran | SelfPay covers 0 | patient pays 18000
DC Kiran | Insurance covers 14400 | patient pays 3600
DC Kiran | Government covers 15000 | patient pays 3000
ER Asha | SelfPay covers 0 | patient pays 6500
ER Asha | Insurance covers 5200 | patient pays 1300
ER Asha | Government covers 6500 | patient pays 0

EXTENSION TESTS
TC Divya: 800
OP Ravi | Corporate covers 1530
IP Meena | Corporate covers 42750
DC Kiran | Corporate covers 16200
ER Asha | Corporate covers 5850
TC Divya | Corporate covers 720
TC Divya | Government rejected: unknown package TC
```

The first part of the output is identical to the output of B1_Billing.cpp, so all the earlier tests still pass. The new bill (TC Divya, 800) is covered 90% by the corporate payer (720). The government scheme has no rate for the new type TC, so it rejects it with a PackageError ("unknown package TC"). This uses the existing exception design without any change.

B4.2 – Check Resource Safety (1 Mark)

The program was compiled and run with:

```
g++ -std=c++17 -Wall -Wextra -g -fsanitize=address B4_Extension.cpp -o b4
./b4
```

The run includes thrown and caught exceptions. The expected result is a clean report: no AddressSanitizer or LeakSanitizer message, and the program ends normally (exit code 0). The error tests of Task B2 were run in the same way and also gave no sanitizer report.

B4.3 – Production Decision (2 Marks)

Decision: the polymorphic design with exceptions is suitable for the hospital group.

Cost: in my measurements (Task B3.2) the virtual version took about 87 ms for 1,000,000 bills, which is about 87 ns per bill. For 200,000 bills a day this is about 17 ms a day, so the extra cost of virtual calls compared with the switch (about 8 ns per bill) is not important for this workload.

Extension effort: with two new bill types and three new payers every year across 15 hospitals, each new type or payer is one new class that can be written and tested separately. No existing class is edited, so the risk of breaking working code is small. The switch design would need the same switch edited in many places every year.

Errors: exceptions are suitable because invalid claims should be a small part of the bills. The error tests showed that every invalid claim is rejected with a clear reason, the objects are destroyed correctly and the sanitizer report is clean. If a large share of the bills were invalid, return codes should be considered. Conditions for production use: keep virtual destructors, use override and noexcept, run the tests under the sanitizer, and measure again with real data.

Alternative Concept and Complexity Analysis

If new bill types and payers were loaded as plug-ins (configuration files describing the rates) instead of being written as new C++ classes, the idea would look like the pseudocode below.

Code / pseudocode of the alternative solution:

```
FILE rates.cfg:
   TC   fee=800
   DL   fee=4000

ALGORITHM TotalFromRules(billType, values)
BEGIN
   SET rule <- LOOKUP billType IN rules table     // loaded from the file at start-up
   IF rule is missing THEN THROW PackageError
   RETURN EVALUATE rule.formula USING values
END
```

Aspect	Original solution	Alternative solution
Time per bill	A virtual call, O(1).	The rule must be looked up and interpreted for every bill, which is slower (a lookup plus parsing or evaluating the rule).
Space for the rules	The rules are in compiled code.	The rules are kept as data in memory (a table of rules for every type).
Effort and risk of extension	A new class must be written, compiled and tested, but the compiler checks it.	No recompiling and a non-programmer can add a rate, but mistakes in the file are found only at run time, so strong validation is needed, and complex rules cannot be described easily.

Conclusion

A new bill type and a new payer were added without changing any existing class, the sanitizer run is expected to be clean, and the measurements support the decision to use the polymorphic design with exceptions.

Overall Learning Outcomes

Through this assignment I learned how to design a program around an abstract interface so that new kinds of bills and payers can be added without changing working code. I understood early and late binding, how the vtable and vptr work, and why a base class needs a virtual destructor. I learned how to design a hierarchy of exception classes, where to catch, log and rethrow an exception, and how stack unwinding and RAII keep a program free of leaks. By testing invalid input, measuring virtual calls against a switch and exceptions against return codes, and running the sanitizer, I learned to judge a design with evidence and not only with a working program.

Final Conclusion

The polymorphic billing system with the exception strategy was designed in Part A and implemented and tested in Part B. Virtual functions and virtual destructors were proved necessary, the error tests showed that invalid data is rejected with clear reasons without losing resources, and the measurements showed that the cost of virtual calls is small for the hospital workload. New bill types and payers can be added as new classes without changing existing code, so the design is suitable for the hospital group.

GitHub Repository Link

Repository link: https://github.com/192524062simats-ops/DSA0103c-/tree/main/c%2B%2B%20assignment

References

[1] Bjarne Stroustrup, The C++ Programming Language, 4th Edition, Addison-Wesley, 2013.

[2] cppreference.com, C++ reference documentation, https://en.cppreference.com

[3] Add your lecture notes, textbook or website references here.

[4]